

Comparative study of supervised machine and deep learning methods for long term temperature prediction in London, United Kingdom and Lima, Peru

NICOLA-ELISABETA ROSCA

STUDENT ID: 33818113

UNIVERSITY OF LONDON, GOLDSMITHS COLLEGE, COMPUTING DEPARTMENT

MSC DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

2023-2024

SUPERVISOR:

DR RAJU CHINTHALAPATI

The code belonging to this project can be found here: <https://github.com/nicolaelisabetarosca/weather-prediction>

Contents

ABSTRACT	1
INTRODUCTION	1
Climate Context.....	1
Weather prediction	2
LITERATURE REVIEW.....	3
Numerical weather prediction	3
Machine learning in weather forecasting	4
METHODOLOGY	6
Dataset presentation	6
Ethical considerations.....	8
Limitations	8
Data pre-processing.....	8
Missing values.....	8
XGBoost date time processing	9
Dummies addition for months.....	9
Converting categorical values to numeric fields using clustering	9
Train-test split	10
Classification.....	10
Forecasting.....	11
Lagged Features	11
XGBoost.....	11
Naïve Forecaster	12
Auto-ARIMA.....	13
SARIMAX	14
LSTM	16
Performance measures	18
Classification	18
Forecasting	18
ANALYSIS	19
Classification	19
Forecasting	20
CONCLUSION	33
FUTURE RECOMMENDATIONS.....	34

ABSTRACT

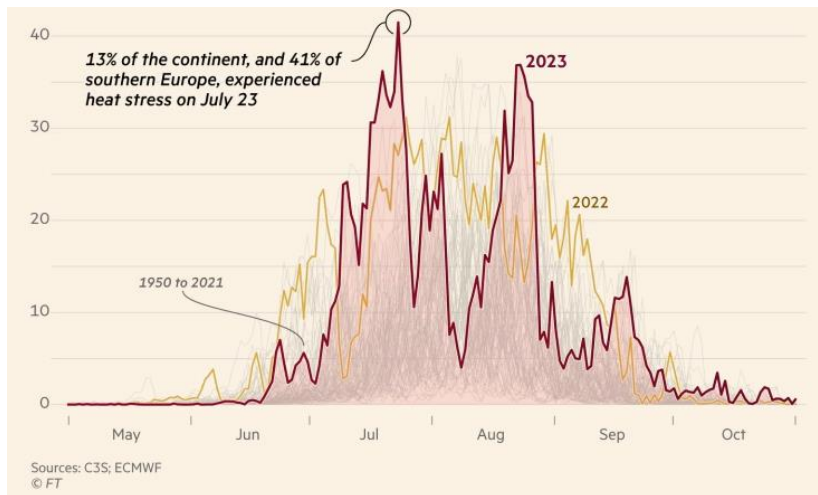
This project is concerned with the forecasting of long-term temperatures in the context of the increased use of machine learning algorithms instead of numerical weather prediction (NWP). London, United Kingdom and Lima, Peru were used as two opposite locations with different weather patterns. The two locations served as the basis of machine learning model comparison between XGBoost, Naïve Forecaster, Auto-ARIMA, SARIMAX and LSTM. This project chose to focus on London, United Kingdom and the temperature as the target, because this location has been experiencing severe heat waves in recent years. In the context of machine learning use in long-term weather forecasting, this project looked at the primary forecast method, numerical weather prediction and its limitations. Then it proved that XGBoost has the highest performance amongst the other models on both locations and therefore should be a primary contender in replacing NWP. The best model had a MAE of 2.1 for the London data and 0.67 for the LIMA data. In the context of weather prediction, this means that the XGBoost model predicted 2 units and respectively 0.67 units away from the truth.

INTRODUCTION

Climate Context

This thesis follows the machine learning process of temperature prediction, using multiple models and evaluating their performance in order to find out which one performs the best. The prediction will be based on nine years' worth of daily temperature data registered in London (UK) and Lima (Peru). The reason behind this comparative study is to verify if machine and deep learning can replace numerical weather prediction in climate modelling, following a series of concerning extreme heat events around the world. If possible, machine learning would be able to have a wider implementation in weather forecasting, providing longer predictions with higher accuracy, without needing extensive meteorological knowledge of atmospheric processes. The research will cover a short context as to why this is needed, the primary method of forecasting at the moment and the implementation of machine and deep learning algorithms on time series data.

Dr Roberta Boscolo, the Climate and Energy lead at the World Meteorological Organization (WMO), often highlights the influence of climate change on weather on her personal LinkedIn profile (*Post | LinkedIn*, 2024), which points the attention towards events that cause high mortality rates and high levels of diseases stemming from climate change, greenhouse gas emissions and unnatural weather events. For example, she shared a Financial Times article (Mooney, 2024) which highlights that “over 70% of the global workforce faced excessive heat in 2020, a 9% increase since 2000. Africa is the hardest hit, with nearly 93% of workers exposed to extreme heat, followed by the Arab states (83.6%) and Asia and the Pacific (74.7%).”, information sourced from the United Nations International Labour Organisation. According to Dr Boscolo, ILO and the WMO, “human-induced climate change is driving global temperatures higher. Regions not traditionally impacted by extreme heat are now experiencing significant changes: Europe and Central Asia: Heat exposure at work increased by 17.3% over 20 years, affecting 29% of workers. More specifically, according to Mooney (2024), Europe has observed record number of days with extreme heat stress in 2023:



Meanwhile, the WMO highlights that heat extremes with temperatures above normal are no longer a “once a year” kind of event. They have increased in frequency and intensity, and it is affecting all aspects of human life as we know it. Which points to just how important weather is to our normal functioning. (*UN Secretary-General issues call to action on extreme heat, 2024*). The temperature of a location is one of the most significant factors in the health of its population, therefore in the context of the number of chaotic meteorological events

increasing, it needs close monitoring. The next part of this introduction is concerned with how weather is monitored and forecasted currently and how it can be improved.

Weather prediction

The weather has been a constant in humanity’s life, therefore the concept of forecasting it is not new and there is plenty of necessity for it in any industry, including economy and healthcare. Weather prediction has been around since before technology. It was done through the observation of factors that contribute to temperature. For example, according to Craig (2011, p. 13-18), historical weather prediction methods included Aristotle’s *Meteorologica* BC, “where he showcased theories based on multiple natural phenomena, from rainbows to snow. Craig (2011) continued to list a few examples of how humanity used to observe the weather and therefore create mechanisms that could predict it, some of them being Nicholas of Cusa which has looked at the moisture absorbed by wool in order to measure humidity (c. 1401 – 64) and Leon Battista Alberti who has invented the anemometer for measuring wind speed (1404) and much more.

The more research in weather observation was done, the more prediction methods came to life. For example, 1940 saw I.A. Kibel developed the first prognostic model by using expansions of the equations of motion for a baroclinic atmosphere in terms of a small parameter. He introduced the quasi-geostrophic approximation with mathematical correctness and calculated the first numerical forecast of the pressure field for Eurasia with the help of an arithmometer” (Tolstykh & Frolov, 2005) . Therefore, Kibel was one of the first to create a numerical weather prediction (NWP) model, which still is the primary method of forecasting to this day. In 2004, the authors showcased that “any modern numerical weather-prediction system consists of a prognostic model responsible for reproducing the dynamics of the atmospheric global circulation and a system assimilating real observational data, which is used to evaluate the current state of the atmosphere”. (Tolstykh & Frolov, 2005)

The next part of this project is the Literature Review, concerned with examples of implementation of machine learning in weather prediction, how they work and what are the most used models so far. The rest of the project is split between Methodology - a discussion of all aspects of the data (source, ethical considerations, variables, etc.), data pre-processing and the models implemented, as well as the performance measures used. Lastly, at the end of the thesis there is the Analysis & Conclusion part, that summarises the reasoning behind the experiment, showcases and explains the results and provides recommendations for further research.

LITERATURE REVIEW

Numerical weather prediction

Ren *et al* (2021) showcase two different ways of forecasting weather. The NWP method is called the theory-driven approach, and the implementation of machine learning is the data-driven approach. Both start from meteorological data but have different ways of applying it, as seen below:

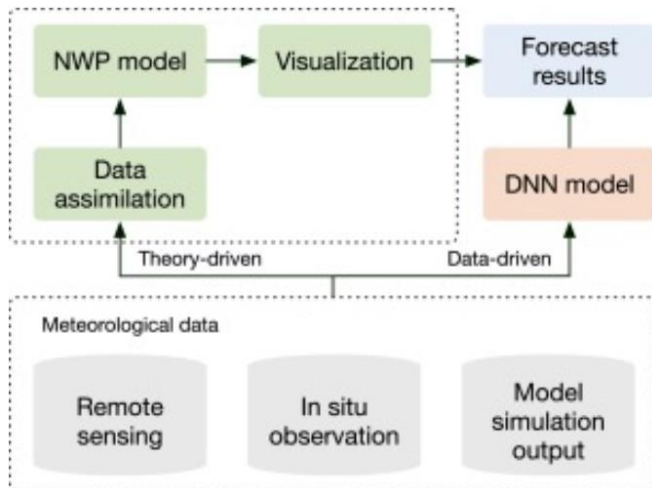


Figure 1. Theory and data driven modelling from Ren *et al* (2021)

Ren *et al* (2021) have highlighted that NWP works with the help of nonlinear partial differentiation equations of a current state which dictates multiple atmospheric processes. The primary process of forecasting weather with the help of NWP is as follows, according to the authors:

- obtaining the original observation datasets, including remote sensing data, in situ observation data, and the simulation results of the NWP models at last time point;
- preprocessing the raw datasets through the data assimilation analysis system, including data quality control and quality assurance;
- inputting the preprocessed datasets into the atmospheric dynamic model equations for prediction; and
- post-processing and visualising the results of prediction (Ren *et al.*, 2021)

Even though this is the primary forecasting method, according to Ren *et al* (2021), “there are still several challenges in NWP. Firstly, due to the chaotic nature of the atmosphere, the small differences in initial conditions have a huge impact on model results”. This means that any new information introduced in the model regarding atmospheric processes will present errors and render the model unusable. This requires more effort and time and it shows a decreased accuracy in the model. The authors continue to highlight that NWP models generate TBs of simulation results each day which are computationally expensive.

Another limitation of the NWP according to Holmstrom *et al* (2016) confirms (Ren *et al.*, 2021) limitation of the NWP, adding that forecasts longer than 10 days become “significantly unreliable”, especially since the initial model is so sensitive to perturbations and uncertainties. This presents the need for a reliable way of producing

true forecasts, with little data and robust enough to take on multiple perturbations. This is regarded to be the theory-driven forecast.

Machine learning can provide all of that without the need of meteorological knowledge, which represents the data driven forecasts. The authors (Ren *et al.*, 2021) found that “weather prediction has been regarded as a physical theory problem, and meteorological scientists have been committed to improving the accuracy of forecasts through the understanding of physical mechanisms, which is a theory-driven approach”. However, data scientists have been working on implementing data-driven deep learning methods of forecasting, therefore not needing the understanding of meteorological aspects.

Machine learning in weather forecasting

Research shows that ML can be both faster and cheaper to execute than regular NWP, and it can also adapt to data much faster when it comes to the chaotic nature of the atmosphere, therefore increasing accuracy. (Ren *et al.*, 2021). This next section will look at the most popular ML models applied in weather prediction.

Firstly, Bochenek and Ustrnul (2022) have conducted a survey of articles that showcase which are the weather elements that are most predicted and which models tend to be the most applied ones. They performed an analysis of 500 relevant scientific articles published since 2018, with their article being published in 2022. They used the popular Google Scholar search engine and looked at the field of climate and NWP. They extracted the most looked at meteorological fields: wind (mentioned 200 times – most popular as it refers to research done in the renewable energy sector), precipitation, temperature, pressure, and radiation. They also found most commonly mentioned methods: Deep Learning, Random Forest, Artificial Neural Networks, Support Vector Machine, and XGBoost, as well as most commonly mentioned countries: China, USA, Australia, India, and Germany. Their conclusion was that supervised learning is the most effective method in the case of climate forecasting, with deep learning being the most used method with a high level of applications in the renewable energy sector. Given that the XGBoost model was ranked the least used model in weather prediction, this thesis will work on implementing it in the model comparison to see if it can provide a good enough long term forecast.

According to Singh *et al* (2019) genetic algorithms and neural networks sometimes “fail to capture the complex relationships between various factors which affect weather”, meaning that the relationship between the atmospheric physical processes is not being “learned”. This shows that NWP could not be replaced by neural networks as of now.

Singh *et al* (2019) describe the work of Abhishek *et al* (2012) where the authors conducted an experiment that predicts the hourly temperature, wind speed and relative humidity 24 hours ahead. Multiple models have been implemented such as a Multilayer Perceptron Networks (MLP), Radial Basis Function Network (RBFN), Autoregression, Autoregressive Moving Average (ARMA), Autoregressive Integrated Moving Average (ARIMA), but they concluded that the recurrent neural networks (RNN) performed better with a time series than classical time series models, including seasonal vector machine (SVM). Additionally, the authors highlight that the prediction window is crucial because the larger the prediction window, the higher the error. Below the comparison of the models show that for a prediction window of 8 weeks, the time series RNN has the smallest RMS error of 1.41 (Singh *et al.*, 2019). In the context of the long term prediction, the time series RNN can be applied to provide the most accurate prediction out of the models mentioned above, which is why this thesis will work on implementing Long Short Term Memory (LSTM) and try to improve a baseline model.

Model	Prediction Window	RMS error
SVM	8 weeks	6.67
ANN	8 Weeks	3.1
Time Series RNN	8 Weeks	1.41

Table 2: comparison of model

Figure 2. Model comparison from Singh et al (2019)

In this article, Ma et al (2020), use artificial intelligence and machine learning to analyse the outdoor temperature and humidity that is used in HVAC (air conditioning) control, for potential identification of the urban heat island effect for a period of 1 to 3 hours. The workflow implemented by Ma et al (2020) follows the same machine learning protocol as this project: load data, data processing, train model, predict temperature and humidity, fit XGBoost to unseen data and parameter update.

Ma et al (2020) showcase multiple examples where neural networks have been used extensively in weather prediction, such as G. Gao et al (2019) have helped reduce the energy use of HVAC through their deep neural network algorithm which predicted the outdoor building environments, including temperature and humidity. Furthermore, Gobakis et al (2011) used a neural network in Athens in summer, and that prediction “has been feeding into the geographic information system application to demonstrate the heat island effect in further few hours”. Lastly, Papantoniou et al (2014) have implemented a neural network used to predict the air temperature in four European cities. According to Ma et al (2020), neural networks have received a lot of attention in terms of weather prediction, which validates the research of Bochenek and Ustrnul (2022) described above in this section. Additionally, the authors also argue that there are multiple other software that could be used for such predictions, however their computing time can be quite extensive.

But Ma et al (2020) have found a method of weather prediction that uses less data as a neural network. They set out to implement the XGBoost model due to its simplicity of use compared to a neural network. In accordance with the authors, XGBoost is more generalised and performs better on different amounts of datasets (even small ones), and “its performance in training set and test set is more consistent” compared to a neural network (Ma et al, 2020).

Their efforts were rewarded with a performance of the XGBoost model of R squared > 0.73 and RMSE < 1.77, which showcase small error rates, therefore proving that the model can easily be implemented in the prediction of temperature and humidity. Their recommendation is for XGBoost to be used to predict more weather factors, such as wind speed, rainfall ration, etc, for a longer period of time of up to 24 hours. (Ma et al, 2020)

This thesis will continue to look at the methodology applied to multiple machine learning and deep learning algorithms that facilitated this comparative study of weather prediction.

METHODOLOGY

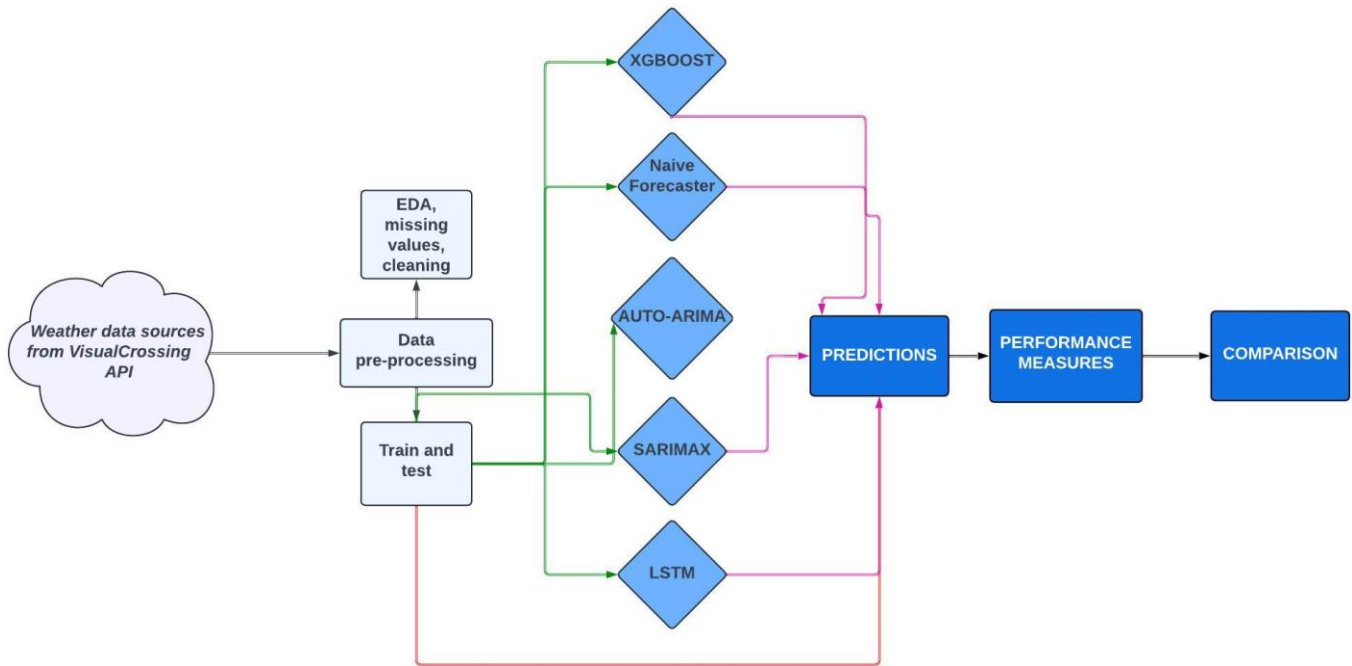


Figure 3. Process of ML implementation specific to this thesis

This is a representation of the machine learning process applied. This next part will follow the methodology applied to the weather data sources from the VisualCrossing API.

Dataset presentation

According to their website, “the Visual Crossing global weather database provides easy access to decades of national and global weather data [...] Our engine processes billions of hourly and sub-hourly weather observations from more than one hundred thousand worldwide observation stations, including satellite and maritime sources to create our global weather observation database” (“Historical Weather Data & Weather Forecast Data | Visual Crossing,” n.d.)

Visual Crossing is a paid API that provides access to the historical data through their web-based data explorer page. According to their website, Visual Crossing is sourcing data from “observation stations across the planet, which is then fed back to services like NOAA and DWD” (“Historical Weather Data & Weather Forecast Data | Visual Crossing,” n.d.). There are plenty of sources to obtain historical weather data from, however it has been difficult to find reliable, independent sources with complete and unskewed data. Visual Crossing was chosen due to complete data and easy to use API.

Requests for the historical weather data have been done through the API requests provided by Visual Crossing. Each link requires the user to include a location, dates and their API key. The response is a JSON (JavaScript Object Notation) format. In this case, the response was the location of London, United Kingdom, with dates from 1st January 2014 to 1st January 2024, which represent a total of 10 years of daily historical weather data plus features, such as dew, humidity, etc. (“Timeline Weather API – Visual Crossing Weather,” 2023)

The API response contains multiple features commonly used in weather forecasting:

Feature	Description
Description	longer text descriptions suitable for displaying in weather displays
Cloudcover	how much of the sky is covered in cloud ranging from 0-100%
Conditions	textual representation of the weather conditions
Datetime	ISO 8601 formatted date, time or datetime value indicating the date and time of the weather data in the local time zone of the requested location
Dew	dew point temperature
Feelslike	what the temperature feels like accounting for heat index or wind chill
Feelslikemax	maximum feels like temperature at the location.
Feelslikemin	minimum feels like temperature at the location
Humidity	relative humidity in %
Icon	a fixed, machine readable summary that can be used to display an icon
Moonphase	represents the fractional portion through the current moon lunation cycle ranging from 0 (the new moon) to 0.5 (the full moon) and back to 1 (the next new moon)
Precip	the amount of liquid precipitation that fell or is predicted to fall in the period
Precipcover	the proportion of hours where there was non-zero precipitation
Precipprob	the likelihood of measurable precipitation ranging from 0% to 100%
Preciptype	an array indicating the type(s) of precipitation expected or that occurred.
Pressure	the sea level atmospheric or barometric pressure in millibars (or hectopascals)
Snow	the amount of snow that fell or is predicted to fall
Snow Depth	the depth of snow on the ground
Source	the type of weather data used for this weather object
Stations	the weather stations used when collecting an historical observation record
Sunrise	The formatted time of the sunrise (For example "2022-05-23T05:50:40").
Sunset	The formatted time of the sunset (For example "2022-05-23T20:22:29").
Temp	temperature at the location. Daily values are average values (mean) for the day.
Tempmax	maximum temperature at the location.
Tempmin	minimum temperature at the location.
Uv Index	a value between 0 and 10 indicating the level of ultraviolet (UV) exposure for that hour or day
Uvindex2	an alternative UV index element that uses the algorithms and models used by the US National Weather Service
Visibility	distance at which distant objects are visible
Wind direction	direction from which the wind is blowing
Wind Gust	instantaneous wind speed at a location
Windspeed	the sustained wind speed measured as the average wind speed that occurs during the preceding one to two minutes.
Windspeedmax	maximum wind speed over the day.
Solar Radiation	(W/m ²) the solar radiation power at the instantaneous moment of the observation

Solarenergy	(MJ /m ²) indicates the total energy from the sun that builds up over an hour or day.
--------------------	---

The target feature in all models was the temperature, and the majority of the features were used in several models as lagged features.

Ethical considerations

Visual Crossing is a business that provides meteorological data for business intelligence and policy making. The historical data that was used for this thesis has been purchased through a monthly individual membership of £35, therefore it is safe to assume that at least some of the funding comes from memberships. Although it is unclear if this is the only funding that the business accepts. They have two headquarters in Virginia, United States and in Germany. At the moment of writing this, there are no known limitations when it comes to sourcing meteorological data from this provider or in general.

Limitations

As with any other research, this thesis was subject to a few limitations. Firstly, the nature of the features used (i.e. natural observations) meant that they had to be shifted by three days. This is problematic because if we were to pick the best model and predict the actual future, we would only be able to do it for the next three days, as that is all that is available in terms of exogenous variables. The comparison of these models was done on the last year of data (test set), because that period counts as historical data and therefore can be used to see if the predictions are correct. Secondly, the features used in this thesis are rather basic in terms of meteorological observations (wind, humidity, etc.), and therefore are not sophisticated. Weather is a highly complex concept and it contains a lot of factors that we cannot codify.

Data pre-processing

Missing values

The given data frames contained a number of missing values from three columns: precipitation type, wind gust and severe risk.

In order to be able to still use the observations from those days, this thesis made use of the k-nearest neighbour method of filling in missing values and chose not to drop those details. According to Zhang (2012), this way of dealing with missing data “are designed according to Minkowski distance or its variants, and have been shown to be generally efficient for numerical variables (features, or attributes)”. The authors present this method of imputation because it is “designed to find k nearest neighbours for a missing datum (incomplete instance) from all complete instances (without missing values) in a given dataset, and then fill in the missing datum with the most frequent one occurring in the neighbours if the target feature (or attribute) is categorical, referred to as majority rule, or with the mean of the neighbours if the target feature is numerical, referred to mean rule [...] Due to its simplicity, easy-understanding and relatively high accuracy, the k-nearest neighbour (kNN) approach has successfully been used in real data processing applications (Chen and Shao, 2000 seen in Zhang, 2012)”.

In shorter terms, the kNN searches through all the given instances, finds the most relevant ones for the missing values and creates new inputs replacing the NaN.

The reason why this thesis chose to use kNN over other methods is that because it is contextual, it performs accurate imputations, without disturbing the relationship between variables.

XGBoost date time processing

Since XGBoost is not a time series predictive model, this project has split the datetime column (day, month and year of the daily observations) into a column for the month and a column for the year in order for the model to be able to differentiate between the two.

Dummies addition for months

The next step was to use the `get_dummies` function from pandas, in order to create twelve columns representing each of the months in one year. According to the pandas documentation (“pandas.get_dummies — pandas 2.2.2 documentation,” n.d.) : “each variable is converted in as many 0/1 variables as there are different values. Columns in the output are each named after a value; if the input is a DataFrame, the name of the original variable is prepended to the value”. This was done in order for the model to learn which month corresponds to which temperature.

Converting categorical values to numeric fields using clustering

The data sourced from Visual Crossing contained one string column titled “description”, which included a series of short sentences describing the weather and temperature on the day. As this was useful information in the context of weather prediction, this project has made use of this column through the tokenization process.

This process converted string data into a numerical representation and made it possible to include this information in the models done afterwards. The tokenization process breaks the strings into features and then we cluster the tokens into groups with the use of the `TfidfTransformer` (“scikit-learn 0.15-git documentation,” n.d.). According to the documentation, “ the goal of using tf-idf instead of the raw frequencies of occurrence of a token in a given document is to scale down the impact of tokens that occur very frequently in a given corpus and that are hence empirically less informative than features that occur in a small fraction of the training corpus”. Each cluster captured sequential patterns which were then represented through word clouds for ease of understanding.

The tokenized data has been summarised into three word clouds, each representing a separate weather state.

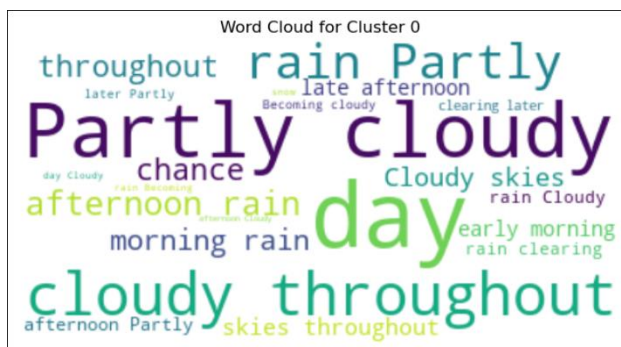


Figure 4. The first cluster (cluster number 0) illustrates a mostly cloudy weather state, however quite rainy.

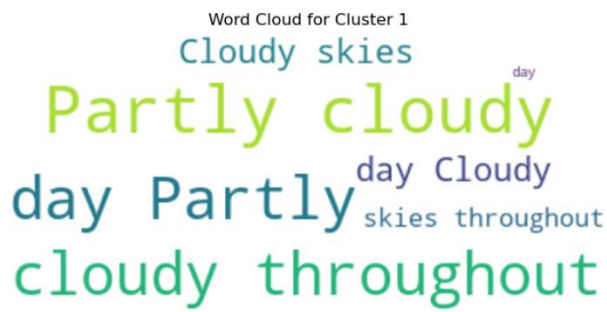


Figure 5. The second cluster, cluster number 1, represents a mostly cloudy day.

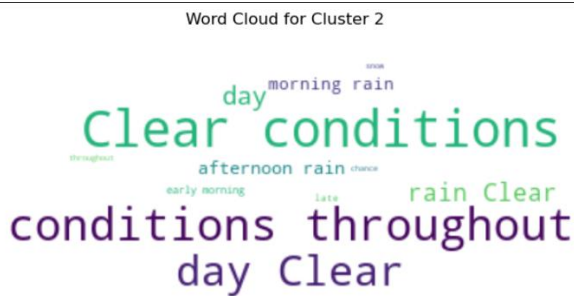


Figure 6. The third cluster, cluster number 2, illustrates a mostly clear day.

Finally, this information is inserted into a column in the main dataframe (“*cluster_description*”), where the data is a mixture 0, 1, or 2, each representing a weather state.

Train-test split

The first model of the project will be XGBoost and the data that is used is time series. The project performed a typical data split for a time series, using the `iloc` function, where it selects all the data from the start up to, but not including, the last 368 rows (days) > training set. Then the test set was created by selecting rows starting from the 368th last row up to the last 3 rows > test set. The final step is to create the data frame for the predictions (*df_future*) that contains the last 3 rows of our original time series (*df_london_final*). The purpose of *df_future* is to hold the features for these 3 days, where predictions will later be generated. At this point, it contains only the features, not the predicted values. This will be utilised in the Lagged Features part of this project.

Classification

Firstly, the data from four locations has been imported into Jupyter Notebook. The Southern hemisphere includes Brisbane, Australia and the Northern Hemisphere includes London (UK), Manila (Philippines) and Bujanovac (Serbia). All locations have been chosen due to their increased number of natural disasters that have taken place in the last ten year (Rosca, 2023).

This part of the project then filtered all four data frames to include all the daily meteorological observations from 1st January 2023.

The data frame used for the training of the model has been initialised as a dataframe with four rows and 1 column. Each location has its own observation row with all the temperatures included in a pandas series, while the target values are initialised as `y_train`. The same process is performed again to initialise the `X_test` and `y_test` series. `X_validation` is created as an empty 2-dimensional labelled data structure with a shape of 2 rows and 1 column. It acts as a pre-allocated dataframe before it is populated with the model classification.

The model used for this hemisphere classification is the `TimeSeriesForestClassifier` from `sktime`. According to its documentation `TfidfTransformer` (“scikit-learn 0.15-git documentation,” n.d.), it is “an ensemble of decision trees built on random intervals”. Each tree has its own sample $\sqrt{m}$ intervals and the mean, standard deviation and slope for each interval and concatenates to form a new dataset. The model ensembles the trees with averaged probability estimates. The model has low configurability, in order to help performance and has a small list of parameters that can be tuned. In this case, the project only used the basic number of estimators (200).

TSF uses a standard decision tree algorithm but with the Entrance gain as the splitting criterion that combines the entropy gain and a distance measure to identify the best splits. As both interpretability and accuracy are required for a classification task, the authors show that TSF can provide both requirements in comparison to other tree based classifiers (Deng et al., 2013).

This experiment came from a place of wanting to understand how a simple observation like the temperature registered in multiple places can help in the identification of the hemisphere. It showed how easy it can be to predict the correct labels with this TSF, but it also shows how it can be implemented on never before seen data. This can contribute massively to situations where one might need to predict locations as part of climate observation.

Forecasting

Lagged Features

Our time series dataset of temperature comes with a large number of features, e.g. humidity, dew, solar radiation, etc In order to utilise these in our forecasting models, we shift them so that the model can learn how the past observations of a given feature impact the target. The chosen lag window for each model is 3 days because it is not so long that it skews the rest of the predictions.

The next section of this project is concerned with the application of multiple machine learning algorithms on the prepared data, generating forecasts, and comparing performance metrics in order to understand which model performed best. Each model has a baseline model and a tuned model. Both types undergo performance metrics that are then compared.

XGBoost

The first model tried in this project was XGBoost, which is an optimised distributed gradient boosting library [...] which implements machine learning algorithms under the Gradient Boosting framework (“XGBoostDocumentation — xgboost 2.1.1 documentation,” n.d.). Gradient boosting is formed out of multiple ensemble algorithms which are constructed from decision tree models. “Each tree is added one at a time to the ensemble and fit to correct the prediction errors made by prior models”, highlights Brownlee (2021). The authors continue to explain that models in the gradient boosting area use an “arbitrary differentiable loss

function and gradient descent optimization algorithm. This gives the technique its name, “*gradient boosting*,” as the loss gradient is minimised as the model is fit, much like a neural network” (Brownlee, 2021).

XGBoost, or Extreme Gradient Boosting, is used widely across multiple industries due to its ease of implementation and range of problems it can adapt to. It also facilitates easy computation and does not require highly specific data processing and it is mostly used for regression predictive modelling (however it can be used as a classifier as well).

Ma et al (2020) that it “can adapt to a large number of cumulative experimental data to improve the performance of the data accumulation model and also have excellent performance in small datasets”. According to the authors, “the basic principle is to combine several low-precision base models into a high-precision model. The loss function of XGBoost has the characteristics of expansibility, high accuracy and good fitting effect. The model of the lifting tree is iterated by the base model to reduce the loss error” (Ma et al, 2020).

Another reason why XGBoost tends to be popular in multiple fields is that due to its boosted trees, it is easy to look at importance scores for each feature that it uses. Each score shows how necessary that feature is in calculating the most accurate prediction. Naturally, not everything is needed in forecasting. In a decision tree, the more valuable a feature is, the higher it is on the tree. According to Brownlee (2016), “importance is calculated for a single decision tree by the amount that each attribute split point improves the performance measure, weighted by the number of observations the node is responsible for. The performance measure may be the purity (Gini index) used to select the split points or another more specific error function. The feature importances are then averaged across all of the decision trees within the model”.

In this project, the baseline model is initialised with the *XGBRegressors()* instance, which will implement the gradient boosting targeted for regression tasks. The model is then fit or “trained”, on a dataset that does not contain the target (the temperature), but does contain all other features, such as *wind speed*, *humidity*, *dew*, *UV index*, etc.

The model then makes a prediction based on the test dataset (unseen data), where the target feature is dropped again. The final step of this first model is to look at the predictions created by it which is an array that provides the sequence of numerical values.

In order to try to improve the model, this project looked at implementing XGBoost grid search, which identifies the optimal parameters for a ML model. The grid search XGB model used parameters such as: the maximum depth of a tree, the learning rate (step size shrinkage used in update to prevent overfitting), the subsample (subsample ratio of the training instances), the maximum leaf nodes and the lambda regulation (L2 regularisation term on weights. Increasing this value will make the model more conservative). (“xgboost 2.1.1 documentation,” n.d.)

The next part of the thesis will look at the Naïve Forecaster model and its grid search.

Naïve Forecaster

The second model used in this project was the Naïve Forecaster from sktime. It is defined as being one of the simplest forms of forecasting, typically used in sales and finance departments as a baseline model. In the context of this project, it utilises the former window of feature and target observations and creates the predictions. As opposed to the former model, XGBoost, the Naïve Forecaster does not use features – it only uses the former target variable and predicts it.

The Naïve Forecaster model does not require complex calculations and can be used as an entry-level model in forecasting. It also does not require too much data as it mostly uses historical data, so it can be used in instances where there is not enough historical information generated. Overall, it can be implemented easily, with little data and used in basic forecasting tasks.

Unfortunately, such an easy model comes with a number of disadvantages. It can hold a major amount of error that can affect the overall analysis. It lacks precise prediction, as it does not take seasonal variation into consideration, or any other patterns nor external factors that can influence the analysis. As it cannot take seasonal variation into consideration, the model can easily underfit or overfit (Sonawane, 2023).

In this project, the Naïve Forecaster was used as a benchmark of all of the models, in order to see how poorly a model can perform with the given data, in the context of climate modelling. The project contains both a baseline model and a grid search.

According to its documentation, the model “obtains the so-called “last window”, a 1D array that denotes the most recent time window that the forecaster is allowed to use”. Its parameters are made out of “*strategy*”, “*sp*” (“seasonal periodicity”) and “*window length*”. This project has made use of the “*strategy*” parameters that come as “*last*”, “*mean*” and “*drift*” (“NaiveForecaster — sktime documentation,” n.d.).

The “*last*” strategy parameter “forecast the last value in the training series when *sp* is 1. When *sp* is not 1, the last value of each season in the last window will be forecasted for each season”. (“NaiveForecaster — sktime documentation,” n.d.)

The “*mean*” strategy parameter “forecast the mean of the last window of the training series when *sp* is 1. When *sp* is not 1, the mean of all values in a season from the last window will be forecasted for each season”. (“NaiveForecaster — sktime documentation,” n.d.)

Finally, the “*drift*” strategy parameter “forecast by fitting a line between the first and last point of the window and extrapolating it into the future”. (“NaiveForecaster — sktime documentation,” n.d.)

The predictions from all strategies are then compared and the best one is added to the data frame of predictions.

Auto-ARIMA

The third model used in this project is the Auto-Regressive Integrated Moving Averages. ARIMA is typically used in instances where a time series analysis is required. It works when the data is stationary (the mean and variance should not vary with time) and when the data used is a univariate series. The model has three core elements:

- AR (auto-regressive term) is the past value used to forecast the next value and it is represented by the *p* value in the PACF plot.
- I (differencing term)
- MA (moving average term) is the “number of past forecast errors used to predict the future values” and is typically represented by the *q* parameters in the ACF plot. (“Time Series Forecasting Methods | Arima In Python and R,” n.d.)

Auto-ARIMA also includes seasonal parameters *P*, *D*, and *Q*, however those are calculated automatically by the model, and do not need user input. Information about these parameters will follow in the next section.

Auto-ARIMA in itself is a grid search of the best combination of parameters given the data and the p and q values. Therefore there was no need to create the ACF and PACF plots at this stage. The model can only be applied to the data once the data is confirmed to be stationary, which was confirmed in the EDA phase of the project through an ADF test. The Augmented Dickey-Fuller test (ADF) is a statistical test that checks the presence of the unit root test in the data, which helps to understand if the data is stationary or not. It has two hypotheses:

- Null Hypothesis: The series has a unit root
- Alternate Hypothesis: The series has no unit root.

```
ADF Statistic: -3.826035220956989
p-value: 0.0026518953035616455
TS is stationary
```

Figure 7. ADF test result showing the London data is stationary

According to its documentation (Perktold et al., 2024), if the “p-value is above a critical size, then we cannot reject that there is a unit root”. In the context of the project, the data was already stationary, therefore the Auto-ARIMA model was applied with no further processing.

There are multiple other advantages to Auto-ARIMA besides the fact that it is easier to implement than its sibling, Arima. According to Purwar (2023), Auto-ARIMA can capture both short term trending and long term trending in a time series analysis. It becomes more efficient and user friendly as it eliminates the manual model tuning.

Unfortunately, the model also struggles in understanding complex data patterns as it assumes that the relationship between future values and past values is almost linear in all scenarios. Additionally, it also can be computationally slow and expensive. Additionally, the model can be sensitive to outliers.

The most outliers were part of the *wind gust* and *solar radiation* features, as seen in the Analysis part of this project, which according to the feature importance done by XGBoost, are quite necessary in correctly predicting the temperature. Therefore, the model could be influenced by the outliers presented.

As with the rest of the models applied on this data, the project started by initialising the Auto-ARIMA instance with the parameters of $sp=12$, which refers to the seasonal periodicity (12 months of the year). The forecaster is then fit to the train data and it starts looking for the best combination of its parameters. The predictions are then done on the test set and added to the data frame prediction data frame.

SARIMAX

The Seasonal Auto Regressive Integrated Moving Average with eXogenous regressors (SARIMAX) is widely used for time series analysis, due to its understanding of the seasonal component and therefore making it the most advanced version of the ARIMA model (Alharbi and Csala, 2022). It is based on the ARIMA model with the addition of the seasonal element and additional features. Besides the p, d, q , SARIMAX also has the P, D , and Q , which according to its documentation (Perktold et al., 2024) are “order of the seasonal component of the model for the AR parameters, differences, MA parameters, and periodicity. D must be an integer indicating the integration order of the process, while P and Q may either be an integers indicating the AR and MA orders (so that all lags up to those orders are included) or else iterables giving specific AR and / or MA lags to include. s is an integer giving the periodicity (number of periods in season), often it is 4 for quarterly data or 12 for monthly data”. This project chose to use SARIMAX instead of SARIMA because it allows exogenous variables, which refers to the most important features that contribute to the prediction of temperature.

As SARIMAX requires the seasonal elements as well, this project made use of the ACF and PACF plots. The seasonal order used for this project was the same as Auto-ARIMA – 12 months.

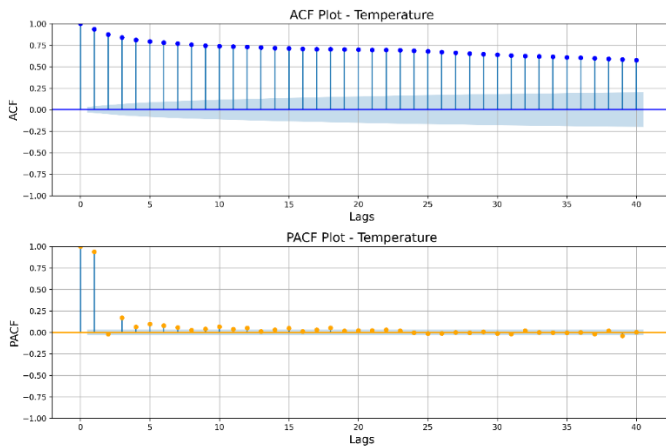


Figure 8. ACF and PACF plots used to look up the SARIMAX parameters

According to Alharbi and Csala (2022), “the SARIMAX model assumes linearity, although the actual temporal connection and covariance are generally non-linear. The regression approaches assume that both the input and output variables follow a Gaussian distribution and that the high level of uncertainty included in the time series data may significantly affect the performance of certain forecasting models. The SARIMAX model has the ability to minimise the error values and enhance the overall accuracy even when the lengths of the input and output dataset are very close to each other and are in similar directions”.

Unfortunately, SARIMAX also comes with some disadvantages, such as complex parameters selection (p, d, q, P, D, Q). This can prove difficult to implement depending on the data, needing more time for processing and thus making it more complex than other models that perform better in a shorter amount of time.

In this project, SARIMAX was implemented by initialising the parameters of endog which is the observed time series target and exog, which represents the array of exogenous regressors (Perktold et al., 2024). They were followed by the order of the p, d, and q parameters and then the seasonal order of P, D, Q and S. The model then was fit to the data and the obtained predictions were added to the predictions data frame.

This project also incorporated a SARIMAX grid search, due to the high number of parameters needed. The grid search was designed to loop through all p, d, q, P, D, Q and S elements and provide all the combinations possible. Then the predictions stemming from all parameters combinations were added to the prediction data frame in order to be able to find the best combination possible via the average results of each combination. The best result came from the ae_0_1_1_0_1_0_12 combination for the London data. The parameters in the best combination of ae_0_1_1_0_1_0_12 are:

- 0 - p
- 1 - d
- 1 - q
- 0 - P

- 1 - D
- 0 - Q
- 12 - S (seasonal)

LSTM

The Long Short Term Memory (LSTM) model is one of the recurrent neural networks (RNN) most popular for large datasets for time series analysis. A RNN is a deep learning model that makes predictions based on sequential inputs : “They are distinguished by their “memory” as they take information from prior inputs to influence the current input and output [...] the output of recurrent neural networks depend on the prior elements within the sequence” (“What Is a Recurrent Neural Network (RNN)?,” 2021).

Some of the advantages of LSTM are incredibly beneficial for time series analysis, such as handling long sequences of information for a longer period of time. LSTMs also include a memory cell “that can store and retrieve information over long sequences. This memory cell allows LSTMs to maintain important information while discarding irrelevant information, making them suitable for tasks that involve remembering past context” (Srivatsavaya, 2023). Finally, they can control the flow of the gradient during backpropagation – the “forget gate, for example, can prevent gradients from vanishing when they need to be propagated back in time. This enables LSTMs to capture information from earlier time steps effectively (Srivatsavaya, 2023).

The main drawbacks of LSTM models include being computationally expensive, as it requires large amounts of data and therefore can make training slower whilst requiring more resources. This model can also overfit when there is not enough data to train on, since the model learns from the previous sequence. If the training data is too short, the model can become confused. The LSTM model is also not the easiest to tune compared to other models in this project – it includes a multitude of parameters that can give different results, whilst using the same data. It can be difficult to implement as it takes a longer period of time to understand which combination of parameters is the best given the analysis at hand.

This part of the project started with a baseline LSTM model with basic parameters, followed by a model with two Dense layers, a model with updated batch size and a final model that includes all the features.

The baseline model was conducted using the *split_sequence()* function which “will split a given univariate sequence into multiple samples where each sample has a specified number of time steps and the output is a single time step” (“How to Develop LSTM Models for Time Series Forecasting - MachineLearningMastery.com,” n.d.). The number of steps used was 30, as it provided the best balance of retaining enough long term memory and not having too short focus on short term memory. Both train and test datasets have been run through the same function.

The baseline model includes one input layer, one hidden layer and one output layer. The model uses the adam optimizer and the *mean_squared_error* loss function. The adam optimizer “ is an optimization algorithm that can be used instead of the classical stochastic gradient descent procedure to update network weights iterative based in training data” (Brownlee, 2017). The *mean_squared_error* loss function is typically used in regression tasks, because it measures the average squared difference between the predicted and actual temperatures. The higher the value, the lower the performance of the model. The baseline model was then run through the *predict_future* function, which takes the model, the data, the number of steps and the predicted number of days as input. The predicted number of days for this project was 365, representing the days in one year.

The next step was to improve the baseline model with additional layers. This time the architecture included as follows: a first input layer, a dropout layer, second layer, second dropout layer, a dense layer, and the output layer. This model differs from the baseline by adding in dense and dropout layers. Dropout is a regularisation method of models that are prone to overfitting. It is used when “input and recurrent connections to LSTM units are probabilistically excluded from activation and weight updates while training a network” (Brownlee, 2017). This dropout was added to the input layer of the LSTM model, for a probability of 20% which means that “the data on the input connection to each LSTM block will be excluded from node activation and weight updates”. The dense layer includes a neuron that obtains results from all the other neurons from the layer used previously, which means that it performs a matrix-vector multiplication, “where the row vector of the output from the preceding layers is equal to the column vector of the dense layer” (“What is Dense Layer in Neural Network?,” n.d.).

The following action was to improve the LSTM model with dense and dropout layers by tuning the batch size. According to Brownlee (2017), “the batch size limits the number of samples to be shown to the network before a weight update can be performed. This same limitation is then imposed when making predictions with the fit model”. Furthermore, the batch size is used to control how many predictions are done at one time.

In the case of this project, the model is using a batch size of 1 and was trained for 3 epochs. A batch size of 1 means that the model always updates the weights after it processes one sample. It leads to frequent updates but each update is using less data than larger batches.

The model was tried with multiple batch sizes, such as 1, 15, 20, 36, 64 etc. The larger the batch size, the more overfitting the model was doing, therefore the batch size was left at 1.

The final stage of the LSTM section was to add the features needed to predict the target. The data went through the *split_sequence()* function, which was defined earlier – it splits the time series data set into sequences needed for sequential data, such as LSTM. The function takes features, targets and *n_steps* as parameters.

The first part required two lists to be initialised that hold space for the sequences of features. The loop goes through the features array and extracts the sequences. It then appends to the lists and converts them to numpy arrays.

The model is then initialised as Sequential, with the architecture being the following:

- First LSTM layer with 100 units which are returned as full sequence for outputs
- A dropout layer of 20%
- Second LSTM layer with 100 units which only returns the output
- Second dropout layer of 20%
- A dense layer with 50 units and a ReLu activation function (Rectified Linear Unit) – “linear function that will output the input directly if it is positive, otherwise, it will output zero. It has become the default activation function for many types of neural networks because a model that uses it is easier to train and often achieves better performance” (Brownlee, 2019).
- The output layer with a single unit added for the prediction of the temperature

As the model outputs a 2D array with a batch size of 1, the next step was to flatten the predictions and add them to the prediction data frame for comparison.

Performance measures

Classification

The performance measures used in the classification issue of this project are accuracy, precision, recall and the F1 score.

- Accuracy: measures how often the classifier correctly predicts. We can define accuracy as the ratio of the number of correct predictions and the total number of predictions.
- Precision: how many of the correctly predicted cases actually turned out to be positive
- Recall: explains how many of the actual positive cases we were able to predict correctly with our model
- F1 score: gives a combined idea about Precision and Recall metrics. It is maximum when Precision is equal to Recall. (Agrawal, 2021)

Forecasting

Each model have had their performance measured calculated as follows:

- MAE (Mean Average Error) – “characterises the alteration among the original and predictable values and is mined as the dataset’s total alteration mean” (Rajawat et al., 2022). Probably the easier performance measure to use in the context of predicting temperature. MAE tells the user how many units away the prediction is from the actual value.
- MSE (Mean Squared Error) – “measures the average squared difference between predicted values and the actual values in the dataset” (“Mean Squared Error | Definition, Formula, Interpretation and Examples,” n.d.)
- RMSE (Root Mean Squared Error) – “the square root of the mean of the square of all of the errors”. It is usually used to compare most machine learning models, “but only to compare prediction errors of different models or model configurations for a particular variable and not between variables, as it is scale-dependent” (“Root-Mean-Square Error in R Programming,” 2020).
- R2 (R squared) – “indicates the percentage of the variance in the dependent variable that the independent variables explain collectively” (Frost, 2017). It is measured on a scale from 0 to 100%.
- MAPE (Mean Absolute Percentage Error) – “relative error measure that uses absolute values to keep the positive and negative errors from cancelling one another out and uses relative errors to enable you to compare forecast accuracy between time-series models” (Oracle documentation, no. 46 in References). This measure was used in order to understand in which direction the forecast goes, i.e. if the predicted temperature is 7, MAPE will be able to say if it is -7 or +7. It is used to address the limitations of the other performance measures.

Each of the models also includes a code that is able to calculate how long it takes for the fit to compute. It is done through the *time* module. At the end of the notebook, there is a data frame called *time_results* that has the name of the model and the corresponding time in seconds.

All models and all code has been implemented on two dataframes, one with data from London (UK) and one with data from Lima (Peru). This was done in order to be able to assess the model’s implementation on data from countries with different weather conditions. The analysis in this project focuses on the London, United Kingdom data and the Lima, Peru analysis is simply for comparison of models.

The details about the requirements of the packages used in this project, alongside the code, can be found at the GitHub link: <https://github.com/nicolaelisabetarosca/weather-prediction>

The next part of the project is concerned with the analysis of the models, the comparison of performance measures, and fit time.

ANALYSIS

Classification

The first part of this thesis demonstrates the hemisphere classification of all four locations chosen: London, Bujanovac, Manila and Brisbane. Taking the temperature time series of the four locations, we trained a time series classifier model which learned the patterns of the training data so that it could predict the class of each series: 1 for the Northern hemisphere and 0 for the Southern hemisphere. The relevance of this classification stems from the hemispheres' significance to the seasonal patterns but also the climate and the direct impact they have over weather - when the Northern Hemisphere is inclined towards the sun, they experience summer. At the same time, the Southern hemisphere is experiencing winter.

The model was fit on the four locations and obtained an accuracy of 100%. Using the fitted model, two new locations' temperature time series were scored, both in the Southern hemisphere: Lagos in Nigeria and Lima in Peru. The predictions on this unseen data shows a 100% accuracy, meaning that each hemisphere has been predicted successfully based on the temperature of each location given.

The performance metrics used were accuracy, precision, recall and the F1 score, all with a result of 1. Below the confusion matrix confirms the results:

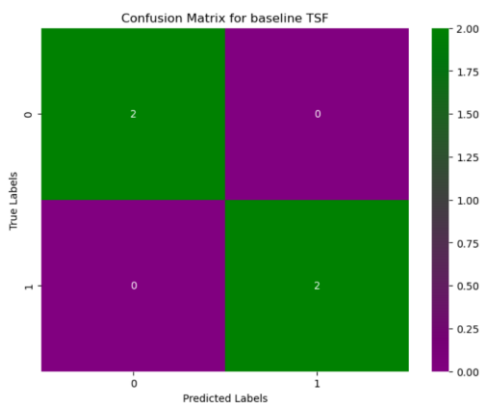


Figure 9. Confusion matrix for hemisphere classification showing an accuracy of 100%

The next part is concerned with the analysis of the forecast models in terms of both accuracy and computing time.

Forecasting

This part of the project has completed an in depth comparative study of various machine learning methods implemented for the temperature of a location. This comparative study has stemmed from a research gap found in the study conducted (Bochenek and Ustrnul, 2022) where they identified that most sources looked at predicting wind for use in the renewable energy industry, with deep learning methods and mostly in China. Therefore, this study has focused on understanding how weather forecasting works and which model is better suited for weather prediction in London (UK), in order to contribute a different perspective to the research.

All models were run first on train data then tested on unseen data. Each model had their performance measures done and put in one dataframe for ease of comparison, as seen below.

	Model	MAE	MSE	RMSE	R2	MAPE
1	XGBoost grid search London	2.103032	7.130397	2.670280	0.741301	-12.492555
0	XGBoost baseline London	2.386807	8.827414	2.971096	0.679731	-7.501623
7	LSTM with two layers London	4.194984	27.362728	5.230939	0.007249	-12.841021
2	Naive Forecaster Baseline London	4.421406	28.217105	5.311978	-0.023749	-44.973731
3	Naive Forecaster Last Strategy London	4.421406	28.217105	5.311978	-0.023749	-44.973731
9	LSTM with features London	4.429446	28.432498	5.332213	-0.031564	-43.502910
4	Auto-ARIMA London	4.440994	27.790480	5.271668	-0.008271	-60.464816
8	LSTM with layers and batch size London	6.326612	59.461907	7.711155	-1.157347	13.045202
10	SARIMAX grid search London	10.032825	126.466961	11.245753	-3.588367	22.349657
5	SARIMAX baseline London	28.006129	1045.717374	32.337554	-36.939833	-431.363625
6	LSTM baseline London	42.981212	2096.584465	45.788475	-75.066503	-606.598101

Figure 10. Model Comparison Results for London Data

	Model	MAE	MSE	RMSE	R2	MAPE
1	XGB grid search Lima	0.671622	0.742997	0.861972	0.845948	0.680950
0	XGB baseline Lima	0.798299	0.998735	0.999367	0.792924	0.860345
4	Auto-ARIMA Lima	1.987309	5.334147	2.309577	-0.105974	-4.322723
2	Naive Forecaster baseline Lima	2.159706	7.951843	2.819901	-0.648724	7.225741
3	naive_gs_lima	2.159706	7.951843	2.819901	-0.648724	7.225741
9	LSTM with features Lima	2.229131	8.378867	2.894627	-0.737262	7.784118
6	LSTM baseline Lima	4.317777	23.775606	4.876024	-3.929599	-13.806992
7	LSTM with two layers Lima	5.945847	40.350587	6.352211	-7.366232	26.656973
8	LSTM with layers and batch size Lima	6.376197	45.461376	6.742505	-8.425896	28.659896
5	SARIMAX baseline Lima	7.251045	88.274745	9.395464	-17.302757	31.904924
10	SARIMAX grid search Lima	18.063132	331.047419	18.194709	-67.638890	84.215832

Figure 11. Model Comparison Results for Lima Data

	Model	Execution Time (seconds)
0	XGBoost Baseline	0.334746
1	XGBoost Grid search	23.837852
2	Naive Forecaster Baseline	0.003000
3	Naive Forecaster Mean strategy	0.134100
4	Naive Forecaster Drift strategy	0.191100
5	Naive Forecaster Last strategy	0.020800
6	Auto-ARIMA	68.547173
7	SARIMAX Baseline	26.651019
8	SARIMAX Best combination	12.568100
9	LSTM Baseline	25.534945
10	LSTM Improved Layers	10.536118
11	LSTM Batch Size	349.397473
12	LSTM with features	19.587304

Figure 12. Computing Time Comparison for London data

XGBoost

The best model for both locations was XGBoost, evidenced by in the images above. The project included both a baseline model and a grid search model and both performed much better than the other models. Both models have used most of the features sourced from the VisualCrossing API. Grid search improved the performance of the models in each case. Grid search loops through all parameters and gives the best combination that can have the most accurate prediction for a longer period of time. The model with the parameters from the grid search had a MAE of only 2.1 (London) and 0.67 (Lima), however we do not know in which direction the prediction went (i.e. over or underpredicting). MAPE was calculated in order to understand the relative size of the error in percentage terms. It tells us that the model tends to underpredict by around -12% (London) and 0.6% (Lima).

Below there is a comparison between the models fit on the two locations:

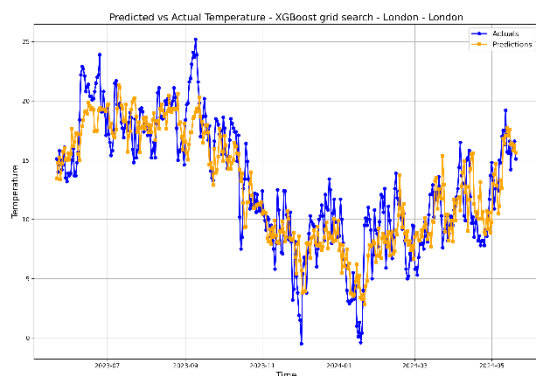


Figure 13. XGBoost for London data

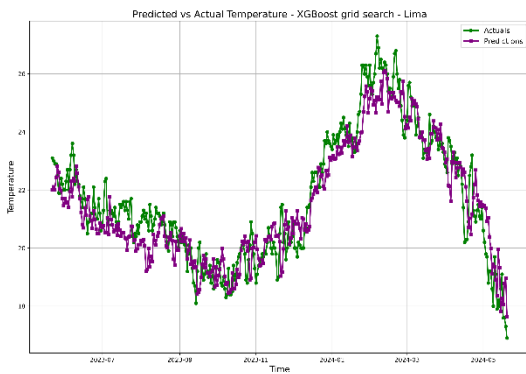


Figure 14. XGBoost for Lima data

The main reason why XGBoost performed this well is because it allows exogenous variables to be part of the prediction (i.e. humidity, dew, UV index, etc.). The more information and context given to the model, the better the prediction will be for a longer period of time. The model understands the complex interactions between different variables more efficiently, which is what is needed in order for machine learning to replace the NWP methods of forecasting. NWP is the primary method for weather forecasting because it implements the atmospheric processes with mathematical calculations, however it seems machine learning proves that it can do the same thing as NWP but with less computational power, easier implementation and not a lot of meteorological knowledge required.

Since it is a tree based model XGBoost also allowed us to look at the feature importance of each variable, as mentioned above.

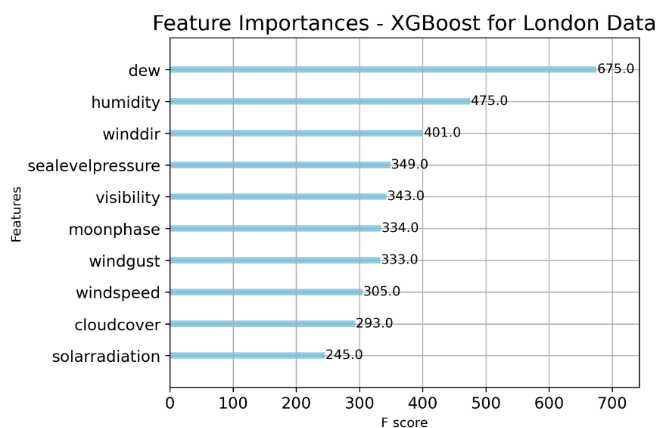


Figure 15. Feature importance for London data

There are multiple factors that can influence the forecast of the temperature, however this graph shows how XGBoost can understand which variable is the most important one based on the location. In this case, the dew point and the humidity are the variables that most impact the temperature in London, United Kingdom. According to the National Weather Service (US Department of Commerce, no 47 in References), “the dew point is the temperature the air needs to be cooled to (at constant pressure) in order to achieve a relative humidity (RH) of 100% [...] The higher the dew point rises, the greater the amount of moisture in the air. This directly affects how “comfortable” it will feel

outside [...] The higher the dew point, the muggier it will feel”. This highlights that the dew, humidity and temperature are directly related and the XGBoost baseline model understood that.

In order to provide a more robust comparison, the same code and the same model has been applied on a location with completely different weather than London. Lima, Peru is part of the Southern hemisphere, and therefore is facing different weather conditions than London. For example, London has snow during winters. This can be seen on the correlation heatmap below.

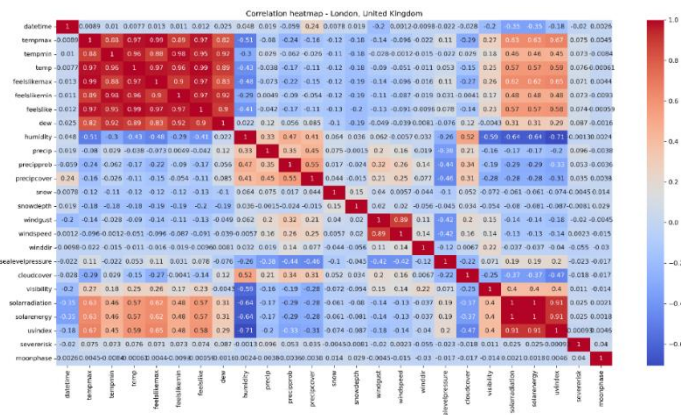


Figure 16. Heatmap London

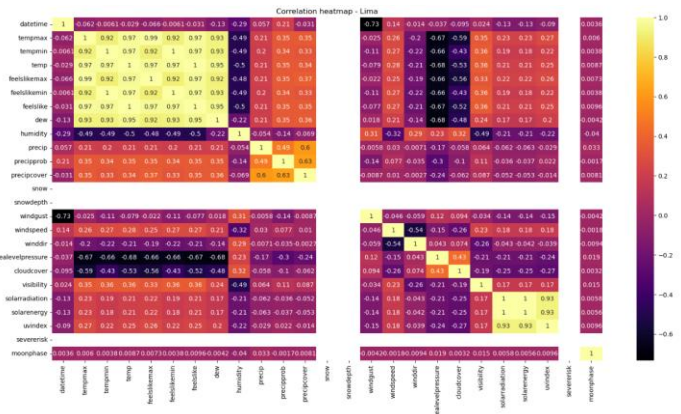


Figure 17. Heatmap Lima

However, Lima does not have snow, and therefore will be treated differently by the model (the columns corresponding to the snow and Snow Depth features are empty). The XGBoost feature importance confirmed that the model is treating features differently, as seen below:

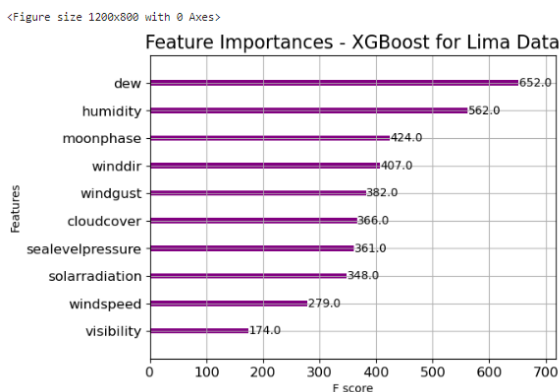


Figure 18. Feature importance for Lima data

Even though this model is not the fastest one, the wait is negligible.

For London and Lima, both dew and humidity are important in the forecasting of weather. However, it seems in Lima, the third most important feature is the moon phase, whereas in London the third most important feature is the wind direction. The model applied on the Lima data showed that it is under 1 units away from the truth.

In terms of computation time, XGBoost was one of the models that had the fastest computation. Unsurprisingly, the grid search model took a bit longer to execute, however it is expected that happens due to the nature of the grid search.

This experiment shows XGBoost’s ability to adapt to different data, making it the primary contender for weather prediction in the context of machine learning vs NWP.

Naive Forecaster

As mentioned in the Methodology, this forecaster does not use other features, just the feature target and it only utilises the former window of feature and target observation to create the predictions. It does not take into consideration seasonal variation or feature interaction, it holds a big amount of error. In a situation where the weather is changing from day to day, be that because of multiple weather factors (dew, humidity, etc.) or due to climate change, a model that only uses the last former window of features and target will overfit or underfit majorly. In this case, the model executed only predicted one temperature for both the London and the Lima data, as seen below in both images:

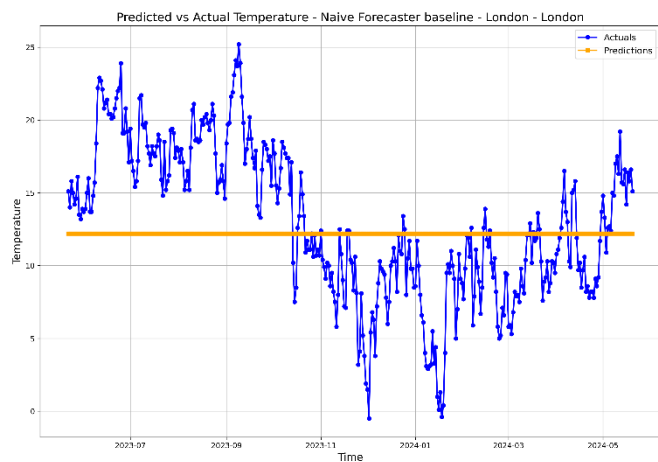


Figure 19. Naive Forecaster prediction on London data

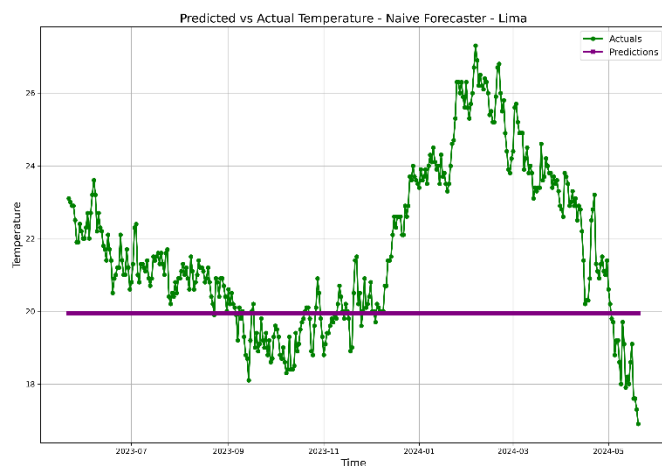


Figure 20. Naive Forecaster prediction on Lima data

The model obtained a MAE of 4.42 and a MAPE of -44 (London) and a MAE of 2.15 and MAPE of 7.2 (Lima). The project focused on trying to improve the forecaster with a grid search, however as it is a basic model it does not have a lot of parameters that can be tuned, only the “strategy” one. The one benefit of the Naïve Forecaster was

that it was executed in record time compared to the rest of the models, however this is due to its simplicity and not due to capability. The London model took 0.004001 seconds and the Lima model took 0.021938 seconds.

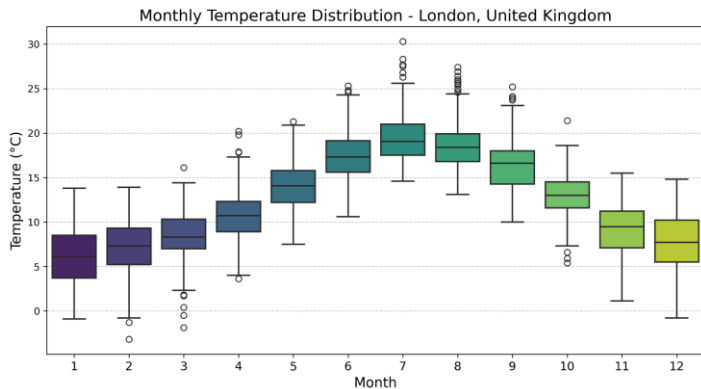


Figure 21. Feature target distribution for London data

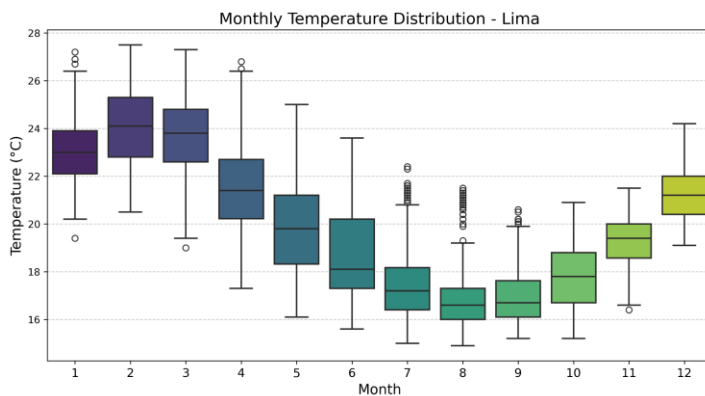


Figure 22. Feature target distribution for Lima data

The distribution of the data has a big impact on the results of the models. In this case, there is a difference between the data distribution for London and Lima. In London, the temperature range is much bigger (4C-30C) than in Lima (15C-28C). The reason why the Naive Forecaster performed quite well on the Lima data could be because the temperature there is more stable, and as the NF model uses the last observation for forecast, it was easier to predict the temperature of the next day. Also, London is known to have several cold snaps or heat waves during the year, and the Naive Forecaster is simply not strong enough to account for that seasonality.

This experiment shows that even though this model is used in industries such as finance as a baseline, it cannot be used for dynamic areas such as weather forecasting.

Auto-ARIMA

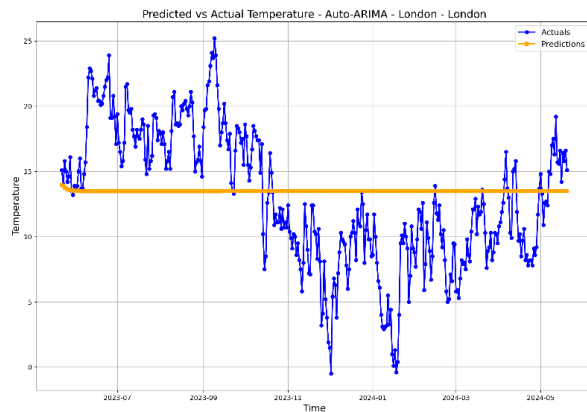


Figure 23. Auto-ARIMA prediction for London data

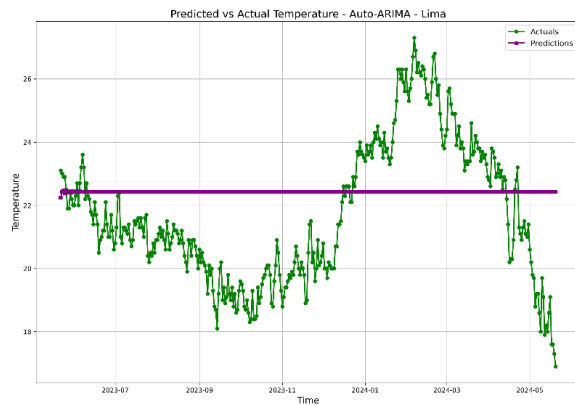


Figure 24. Auto-ARIMA prediction for Lima data

As seen in the above images, the model started to fit onto the data however it lost its progress as the forecast got longer. It shows that Auto-ARIMA in this case could potentially perform well on very short forecasts with the right data. As mentioned in the methodology, the longer the prediction window, the worse the prediction results.

The methodology of this thesis also showed that the model has a few disadvantages that damage the results. Firstly, it struggled to understand the data patterns that come with weather prediction, especially since the temperature has been greatly influenced by recent climate change issues. It assumes that the relationship between future values and past values is almost linear. It is also greatly influenced by outliers, and there are quite a few in both locations:

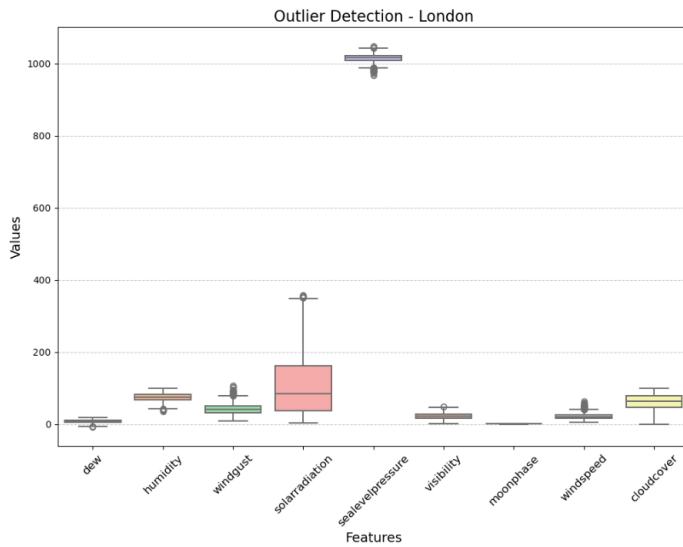


Figure 25. Outlier detection for London data

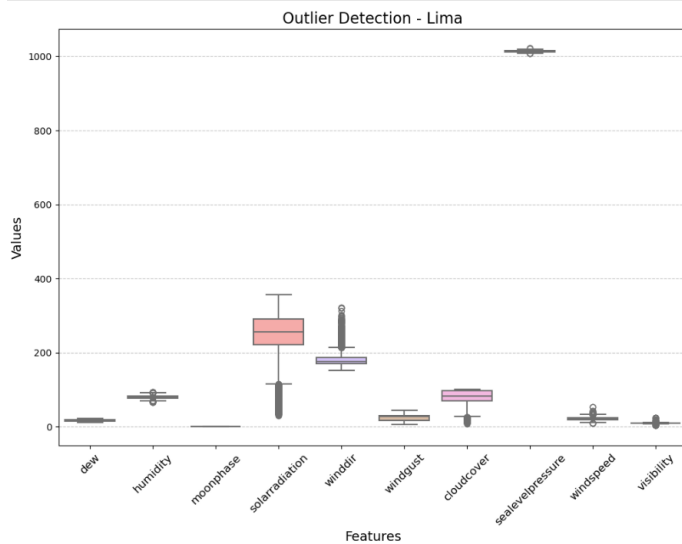


Figure 26. Outlier detection for Lima data

Both locations have outliers in the sea level pressure, solar radiation, cloud cover features, with additional outliers in the wind direction feature in Lima. All mentioned features have an immense impact on the prediction of the temperature as seen in the feature importance images above. The model's performance measured a MAE of 4.44099 and a MAPE of -60.464816. The model also took 72.924164 seconds to compute.

This experiment showed that the Auto-ARIMA performs well at the start of a prediction, however it does not carry that through the entire prediction window. Unfortunately, this model also does not measure up to the XGBoost.

SARIMAX

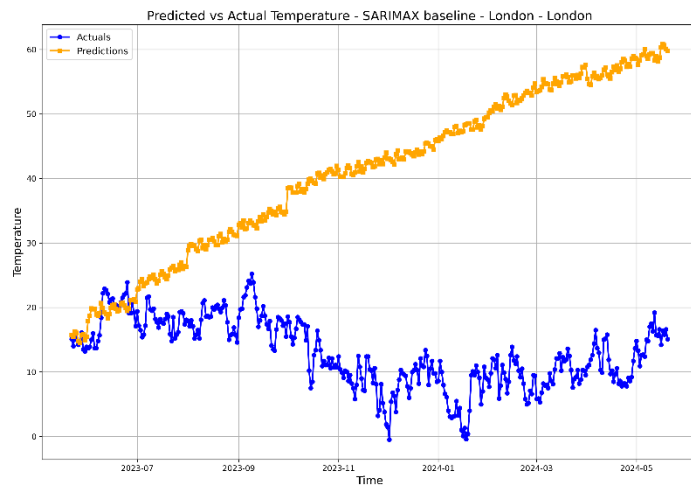


Figure 27. SARIMAX predictions on London data

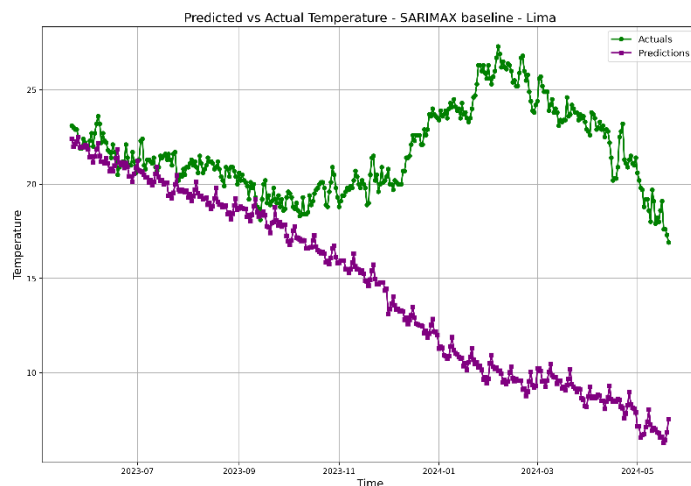


Figure 28. SARIMAX predictions on Lima data

As it has the seasonal component implemented, the SARIMAX baseline model performed much better than Auto-ARIMA, however still did not do well enough to surpass XGBoost's performance. The baseline model had a MAE of 28.006129 and a MAPE of -431.363625, however the grid search model had a MAE of 10.032825 and a MAPE of 22.349657 for the London data.

On both locations the models either overfit or underfit, however the forecast lines for both locations show that the models have been able to pick up the daily trend, which is expected as it used exogenous variables. As mentioned in the Naive-Forecaster analysis, the model would perform better than SARIMAX, because the jump from mean to mean is smaller, and therefore the NF predictions will always be closer to the actual values than the SARIMAX model which underfit and overfit respectively.

The baseline model took 36.845071 seconds to compute, but the grid search took 12.568100 seconds for the best combination, which was unexpected. The expectation was that a baseline model would be computed

faster, as the grid search goes through multiple combinations at once. However, this shows that SARIMAX's strength relies on grid search.

This shows that SARIMAX, if improved, could potentially be one of the machine learning methods to surpass NWP, however it was not strong enough to do it at the time of writing this.

LSTM

LSTM Baseline

The last model of the project was LSTM, which has been improved with additional layers, dropout, batch size and the addition of features. The baseline LSTM model performed well enough for a model with no tuning and no features:

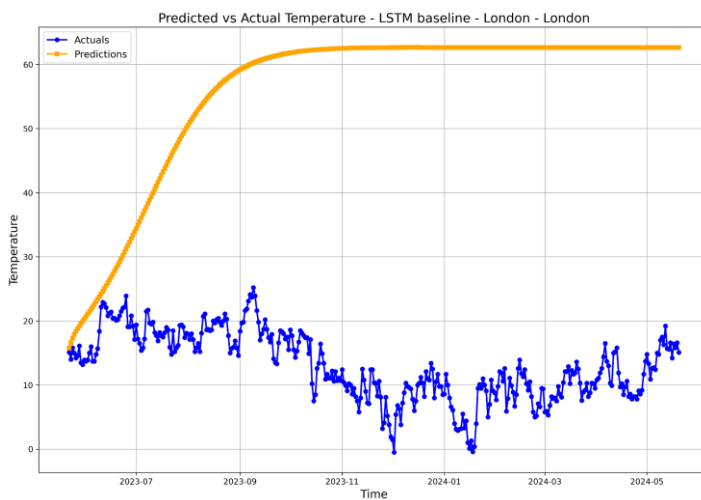


Figure 29. LSTM baseline predictions on London data

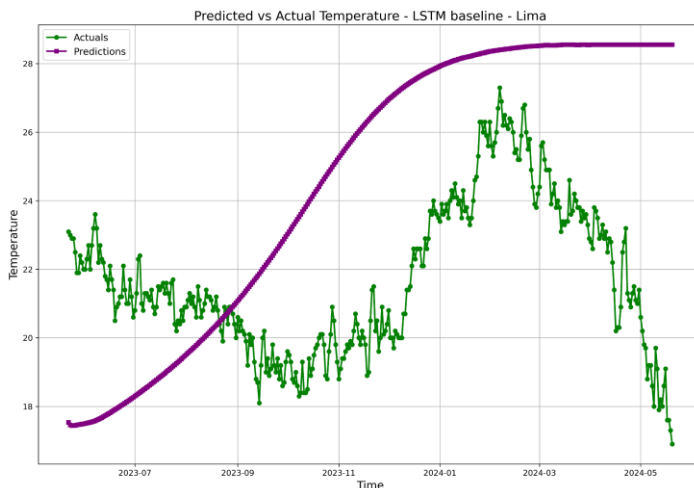


Figure 30. LSTM baseline prediction on Lima data

Even though it did not pick up on the trend of the daily data in either of the locations, it did pick up on the direction of the predictions for the London data. The orange line shows that it overfit significantly and this is reflected in the performance measures. It had an MAE of 42.981212 and MAPE of -606.598101. The baseline model took 25.534945 seconds to compute.

For the Lima data however, the purple line shows that the model was slowly following the height of the data starting with 2023-01. The model performed much better than the London model, with a MAE of 4.3177 and a MAPE of -13.806992.

Keeping in mind that data from both Locations have the same features, same time stamps and the same data engineering and split, the inconsistency shown by the LSTM baseline models highlights that it cannot be used for long term prediction. Firstly, because the model needs to be tailored to a specific data and secondly, because it requires too much time to implement correctly.

LSTM with additional layers and dropout

The next part saw LSTM fit the data with additional layers and dropout layers:

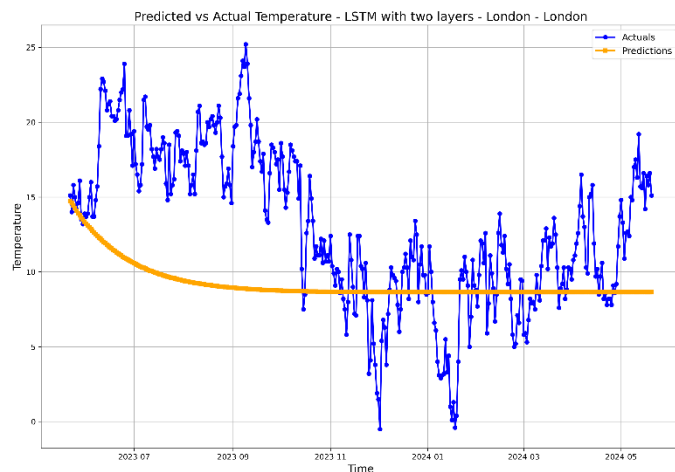


Figure 31. LSTM layers on London data

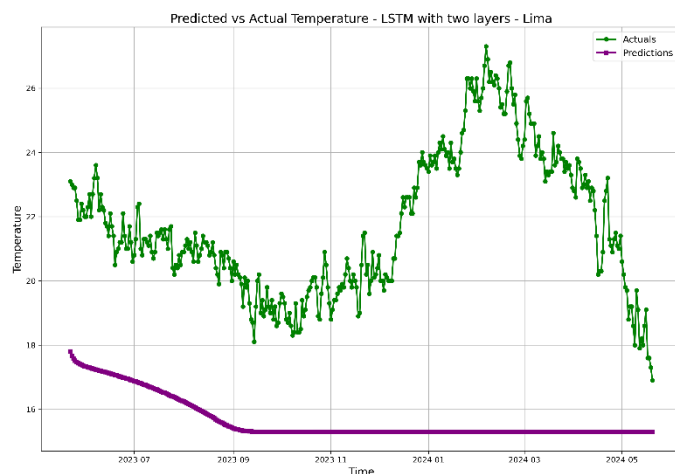


Figure 32. LSTM layers on Lima data

LSTM with layers came with a MAE of 4.199541 and a MAPE of -12.841021 for the London data. For the Lima data, the model had a MAE of 5.945847 and a MAPE of 26.656973. The London model started to understand the data and the relationship between variables at the start, however not enough to be able to fit the data properly. The Lima model underfitted from the start. The London model took 10.536118 seconds to compute which shows the work that the additional layers do in improving the model, as it took 15 seconds less than the baseline model.

Out of the two locations, the London model performed better, however not enough to be considered for the replacement of NWP in weather prediction.

LSTM with batch size

Therefore, the next model was LSTM with additional layers and an improved batch size of 1:

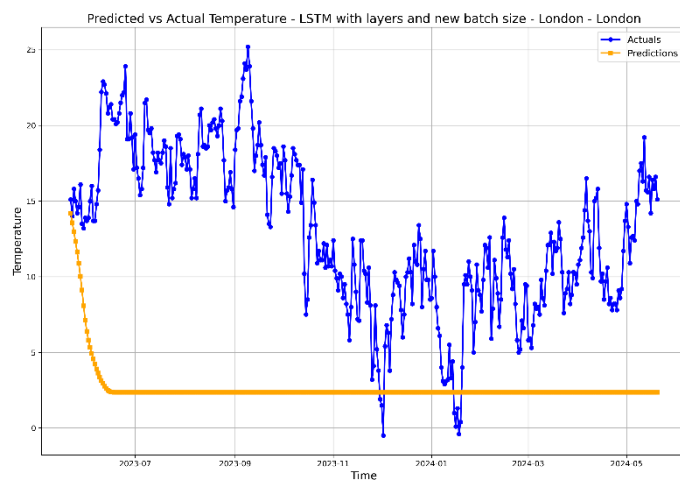


Figure 33. LSTM dropout on London data

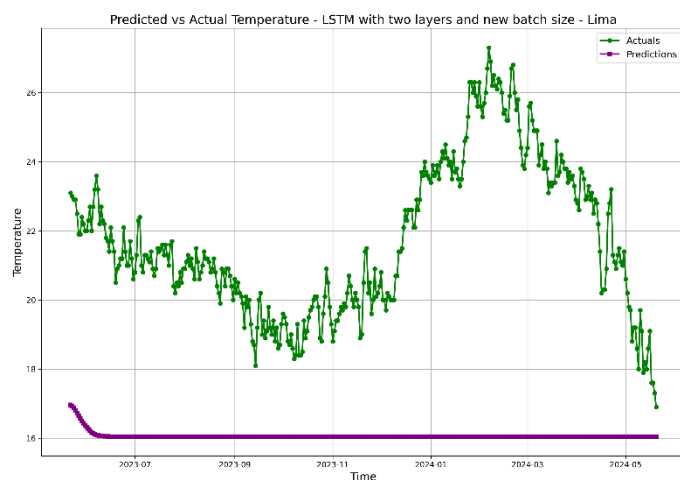


Figure 34. LSTM dropout on Lima data

The model for the London data can be seen to pick up on the daily temperature trend better than the baseline models and better than the Lima model. The MAE for the London model was 6.32661 and the MAPE was

13.045202. The model also took 349.397473 seconds to compute, which makes it the model with the highest time in this project.

The Lima mode had an MAE of 6.376197 and a MAPE of 28.659896, and took 174.867840 seconds to compute.

Neither models performed well enough, and there is no winner between them either. The batch size was difficult to tune, because the higher it was, the longer it took to compute. The batch size improves the model significantly as it allows it to pick up on the data trend, however it is not enough.

LSTM with features

The next and final step was to try to improve the LSTM with the addition of exogenous variables:

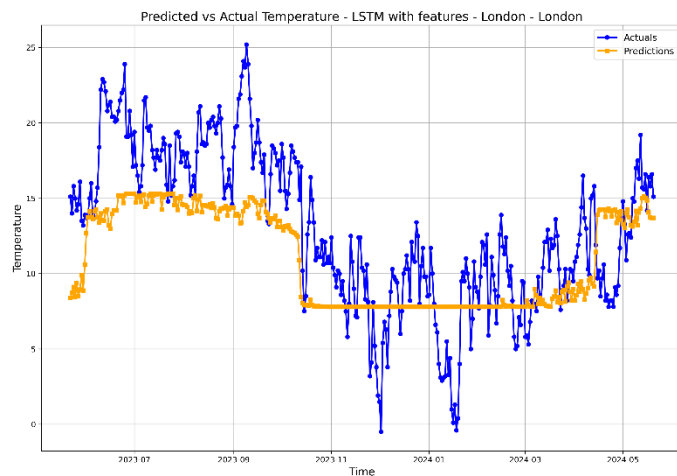


Figure 35. LSTM features on London data

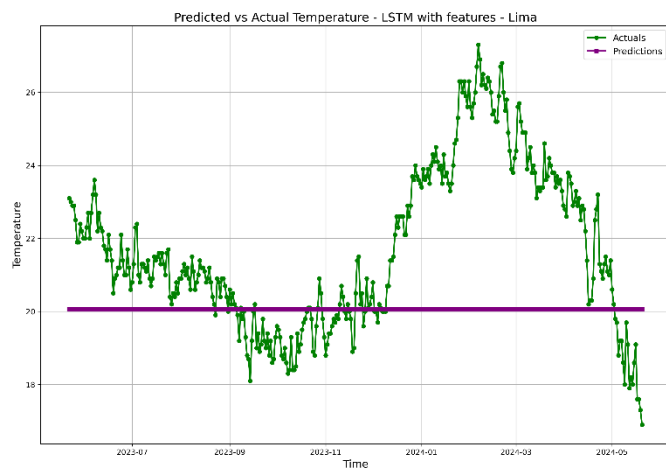


Figure 36. LSTM features on Lima data

It seems that for both locations, although introducing the features helped the model to predict closer to the mean of the data points, it is still unable to pick up the lower-level granularity of the temperature changes, as seen below:

	temp	lstm_features
2023-05-22	15.1	12.066167
2023-05-23	14.0	12.066167
2023-05-24	15.8	12.066167
2023-05-25	15.0	12.066167
2023-05-26	14.2	12.066167
...	...	...
2024-05-16	14.2	12.066234
2024-05-17	16.4	12.066234
2024-05-18	15.8	12.066234
2024-05-19	16.6	12.066234
2024-05-20	15.1	12.066234

Figure 37. LSTM features predictions showcasing that the model predicted the same temperature for all months and years

The image above represents the predictions for the London data, however this applies to the Lima data as well. It seems that neither models were able to understand the relationships between variables, which is understandable because LSTM is not a tree based model. As a recurrent neural network, LSTM is mostly designed to use sequential data for time series, not to understand the variable relationships. Equally, due to its black box interpretation, it is difficult to understand which variable had the biggest contribution.

This model had a MAE of 4.429446 (London) and MAPE of -43.502910 (London). For Lima, it had a MAE of 2.229131 and a MAPE of 7.784118. The model performed slightly better on the Lima data, and given the fact that the XGBoost model was only 2 units away from the truth, one might say that the LSTM with features performed well enough. However, for the Lima data, the XGBoost model predicted under 1 unit away from the truth, which still makes it the best result in this comparative study.

Another possible limitation of LSTM in this project might have stemmed from the small amount of data used and the low dimensionality. As mentioned previously, the train data used is 9 years' worth of daily temperatures and the test data is 1 year. For most models, this would be enough data to be able to create good enough predictions, however as LSTM is sequential, it most likely required over 10 years' worth of data in order to provide better results, with a feature set of higher dimension.

Finally, the result of this comparison study was XGBoost grid search for both locations. The best model was fitted onto the Lima data, however this thesis showed that the model can easily fit on multiple locations' data with minimal location-specific data engineering, whilst providing fairly accurate results.

CONCLUSION

In conclusion, this project completed two tasks: a hemisphere classification and a forecast of weather in London, United Kingdom and Lima, Peru. The motivation behind this comparison stemmed from the increase

of “one of a kind” heatwaves that London has been facing since 2020. Every year, in most continents, each month goes above the average of said month the previous year. It seems that humanity is on a trajectory path of severe heat and severe cold – all weather is heading to extremes, with no balance. In order to save lives and property, policy making around climate change needs to be updated to the current crisis. To do that, we need to look at the limitations of the current methods of prediction.

NWP is the primary method of forecasting at the moment, requiring big computational time, mathematical operations and in-depth knowledge of atmospheric and meteorological processes. It takes a lot of effort, processing power and time and knowledge to be able to create an accurate long-term forecast for a location. Even then, as the climate crisis is taking its toll on the weather, multiple extreme events will render the forecast useless. However, machine learning has been able to “understand” the relationship between variables without knowledge of meteorological processes, it takes less computation time and the data engineering is minimal if the right models are being used.

This project has been concerned with a comparison of some of the common time series models available. According to (Bochenek and Ustrnul, 2022), most of the machine learning models have been used for the prediction of wind for applications in the renewable energy industry, mostly done in China and mostly focusing on deep learning. XGBoost seemed to be at the bottom of the list, even though it requires minimal data engineering, it is fast and efficient and can understand trends in data and the relationship between variables. Therefore, it was the first model tried, alongside Naive Forecaster, Auto-ARIMA, SARIMAX and LSTM.

Even though most of the remaining models are made specifically for time series data, with seasonal components built in, XGBoost performed the best, providing a prediction of 2 units away from the truth. In order to see if this would be a one-time-only situation, the exact same model with the exact same data processing has been applied on data from a location with completely different weather patterns than London – Lima, Peru. As it is in a different hemisphere, Lima experiences different weather, for example it does not have snow during winter. The model performed even better on the Lima data, where it provided a forecast of only 0.6 units away from the truth.

Overall, this project showcased the high level of adaptation of the XGBoost model and hopefully it provided new evidence as to why more attention should be paid to it in the context of machine learning versus NWP weather forecasts.

FUTURE RECOMMENDATIONS

Given the strong performance of the XGBoost model, the future recommendation is to implement XGBoost for weather monitoring on continuous incoming data. Once the model is set up, it can create continuous forecasts and once an extreme weather event is flagged in the “future”, those working in anticipatory action for natural disasters can take action to protect both lives and property well ahead of time.

The second recommendation would be the improvement of LSTM with exogenous variables. The model showed that it is capable of capturing the trend of the temperature long term, however since it has multiple parameters worth tuning, this project only focused on layers and batch size. It would be worth looking into other parameters such as activation functions and learning rate, or momentum – “unique hyperparameter which allows the accumulation of the gradients of the past steps to determine the direction to go with, instead of using the gradient of only the current step to guide the search” (“10 Hyperparameters to keep an eye on for your LSTM model — and other tips | by Kuldeep Chowdhury | Geek Culture | Medium,” n.d.).

The third recommendation would be to apply XGBoost on a variety of observations such as satellite images or multi-dimensional data. It would be useful to understand if XGBoost would be able to perform as well on satellite data, for better visualisation and better understanding of the risks stemming from climate change.

REFERENCES

The bibliography is organized in the order in which the references appear throughout the thesis.

1. Robert Boscolo, Post | LinkedIn [WWW Document], n.d. URL https://www.linkedin.com/posts/roberta-boscolo-89247216_climatechange-heatwaves-activity-7222520364076072961-sV-O/?utm_source=share&utm_medium=member_desktop (accessed 8.30.24).
2. Mooney, A., 2024. Workers in Europe and central Asia suffer jump in heat-related stress. Financial Times.
3. UN Secretary-General issues call to action on extreme heat [WWW Document], 2024. . World Meteorological Organization. URL <https://wmo.int/media/news/un-secretary-general-issues-call-action-extreme-heat-0> (accessed 8.30.24).
4. Craig, D., 2011. The Weather Book.
5. Tolstykh, M., Frolov, A., 2005. Some Current Problems in Numerical Weather Prediction. *Izvestiya Atmospheric and Oceanic Physics* 41, 285–295.
6. Ren, X., Li, X., Ren, K., Song, J., Xu, Z., Deng, K., Wang, X., 2021. Deep Learning-Based Weather Prediction: A Survey. *Big Data Research* 23, 100178. <https://doi.org/10.1016/j.bdr.2020.100178>
7. Holmstrom, M., Liu, D., Vo, C., 2016. Machine Learning Applied to Weather Forecasting. <https://cs229.stanford.edu/proj2016/report/HolmstromLiuVo-MachineLearningAppliedToWeatherForecasting-report.pdf>
8. Bochenek, B., Ustrnul, Z., 2022. Machine Learning in Weather Prediction and Climate Analyses—Applications and Perspectives. *Atmosphere* 13, 180. <https://doi.org/10.3390/atmos13020180>
9. Missing info
10. Singh, S., Kaushik, M., Gupta, A., Malviya, A.K., 2019. Weather Forecasting Using Machine Learning Techniques. <https://doi.org/10.2139/ssrn.3350281>
11. Abhishek, K., Singh, M.P., Ghosh, S., Anand, A., 2012. Weather Forecasting Model using Artificial Neural Network. *Procedia Technology*, 2nd International Conference on Computer, Communication, Control and Information Technology(C3IT-2012) on February 25 - 26, 2012 4, 311–318. <https://doi.org/10.1016/j.protcy.2012.05.047>
12. Ma, X., Fang, C., Ji, J., 2020. Prediction of outdoor air temperature and humidity using Xgboost. *IOP Conf. Ser.: Earth Environ. Sci.* 427, 012013. <https://doi.org/10.1088/1755-1315/427/1/012013>
13. Gao, G., Li, J., & Wen, Y. (2019) ‘Energy-efficient thermal comfort control in smart buildings via deep reinforcement learning’, *arXiv*, 1901.04693v1
14. Gobakis, K., Kolokotsa, D., Synnefa, A., Saliari, M., Giannopoulou, K. & Santamouris, M. (2011) ‘Development of a model for urban heat island prediction using neural network techniques’, *Sustainable Cities and Society*, 1(2), pp. 104–115.
15. Papantoniou, S. & Kolokotsa, D. (2014) ‘Prediction of outdoor air temperature using neural networks: application in 4 European cities’, *Third International Conference on Countermeasures to Urban Heat Island*.

16. Historical Weather Data & Weather Forecast Data | Visual Crossing [WWW Document], n.d. URL <https://www.visualcrossing.com/weather-data> (accessed 9.1.24).
17. Timeline Weather API – Visual Crossing Weather, 2023. URL <https://www.visualcrossing.com/resources/documentation/weather-api/timeline-weather-api/> (accessed 9.5.24).
18. Zhang, S., 2012. Nearest neighbor selection for iteratively *k*NN imputation. *Journal of Systems and Software* 85, 2541–2552. <https://doi.org/10.1016/j.jss.2012.05.073>
19. Chen, J., Shao, J., n.d. Nearest Neighbor Imputation for Survey Data.
20. pandas.get_dummies — pandas 2.2.2 documentation [WWW Document], n.d. URL https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html (accessed 8.30.24).
21. sklearn.feature_extraction.text.TfidfTransformer — scikit-learn 0.15-git documentation [WWW Document], n.d. URL https://scikit-learn.org/0.15/modules/generated/sklearn.feature_extraction.text.TfidfTransformer.html (accessed 9.5.24).
22. Rosca, N. (2023) ‘The impact of natural disasters on the European population: An analysis of the natural disasters that took place between 2013 and 2023 in Eastern and Southern Europe’, Unpublished coursework, Data Programming module, MSc Data Science and AI, Goldsmiths University of London.
23. Deng, H., Runger, G., Tuv, E., Vladimir, M., 2013. A Time Series Forest for Classification and Feature Extraction.
24. XGBoost Documentation — xgboost 2.1.1 documentation [WWW Document], n.d. URL <https://xgboost.readthedocs.io/en/stable/> (accessed 8.30.24).
25. Brownlee, J., 2021. XGBoost for Regression. *MachineLearningMastery.com*. URL <https://machinelearningmastery.com/xgboost-for-regression/> (accessed 8.30.24).
26. Brownlee, J., 2016. Feature Importance and Feature Selection With XGBoost in Python. *MachineLearningMastery.com*. URL <https://machinelearningmastery.com/feature-importance-and-feature-selection-with-xgboost-in-python/> (accessed 8.30.24).
27. XGBoost Parameters — xgboost 2.1.1 documentation [WWW Document], n.d. URL <https://xgboost.readthedocs.io/en/stable/parameter.html> (accessed 9.3.24).
28. Sonawane, R., 2023. Pros and Cons of Naïve Forecasting in Supply Chain Management. URL <https://transimpact.in/nextsights/naive-forecasting-in-supply-chain-management-pros-and-cons/> (accessed 9.5.24).
29. NaiveForecaster — sktime documentation [WWW Document], n.d. URL https://www.sktime.net/en/stable/api_reference/auto_generated/sktime.forecasting.naive.NaiveForecaster.html (accessed 9.5.24).
30. Time Series Forecasting Methods | Arima In Python and R [WWW Document], n.d. URL <https://www.analyticsvidhya.com/blog/2018/08/auto-arima-time-series-modeling-python-r/> (accessed 9.5.24).
31. Josef Perktold, Skipper Seabold, Kevin Sheppard, ChadFulton, Kerby Shedden, jbrockmendel, j-grana6, Peter Quackenbush, Vincent Arel-Bundock, Wes McKinney, Ian Langmore, Bart Baker, Ralf Gommers, yogabonito, s-scherrer, Yauhen Zhurko, Matthew Brett, Enrico Giampieri, yl565, Jarrod Millman, Paul Hobson, Vincent, Pamphile Roy, Tom Augspurger, tvanzyl, alexbrc, Tyler Hartley, Fernando Perez, Yuji Tamiya, Yaroslav Halchenko, 2024. statsmodels/statsmodels: Release 0.14.2. <https://doi.org/10.5281/ZENODO.593847>
32. ARIMA Model. Understanding ARIMA Model | by Siddhartha Purwar | Medium [WWW Document], n.d. URL <https://siddp6.medium.com/arima-model-424a1b0f0970> (accessed 9.5.24).

33. Alharbi, F.R., Csala, D., 2022. A Seasonal Autoregressive Integrated Moving Average with Exogenous Factors (SARIMAX) Forecasting Model-Based Time Series Approach. *Inventions* 7, 94.
<https://doi.org/10.3390/inventions7040094>
34. What Is a Recurrent Neural Network (RNN)? | IBM [WWW Document], 2021. URL <https://www.ibm.com/topics/recurrent-neural-networks> (accessed 9.5.24).
35. Srivatsavaya, P., 2023. LSTM — Implementation, Advantages and Diadvantages. Medium. URL <https://medium.com/@prudhviraju.srivatsavaya/lstm-implementation-advantages-and-diadvantages-914a96fa0acb> (accessed 9.5.24).
36. How to Develop LSTM Models for Time Series Forecasting - MachineLearningMastery.com [WWW Document], n.d. URL <https://machinelearningmastery.com/how-to-develop-lstm-models-for-time-series-forecasting/> (accessed 9.5.24).
37. Brownlee, J., 2017. Gentle Introduction to the Adam Optimization Algorithm for Deep Learning. MachineLearningMastery.com. URL <https://machinelearningmastery.com/adam-optimization-algorithm-for-deep-learning/> (accessed 9.5.24).
38. Brownlee, J., 2017. Dropout with LSTM Networks for Time Series Forecasting. MachineLearningMastery.com. URL <https://machinelearningmastery.com/use-dropout-lstm-networks-time-series-forecasting/> (accessed 9.5.24).
39. What is Dense Layer in Neural Network? [WWW Document], n.d. URL <https://analyticsindiamag.com/topics/what-is-dense-layer-in-neural-network/#h-what-is-a-dense-layer> (accessed 9.5.24).
40. Brownlee, J., 2017. How to use Different Batch Sizes when Training and Predicting with LSTMs. MachineLearningMastery.com. URL <https://machinelearningmastery.com/use-different-batch-sizes-training-predicting-python-keras/> (accessed 9.5.24).
41. Brownlee, J., 2019. A Gentle Introduction to the Rectified Linear Unit (ReLU). MachineLearningMastery.com. URL <https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/> (accessed 9.5.24).
42. Rajawat, A.S., Mohammed, O., Shaw, R.N., Ghosh, A., 2022. Chapter six - Renewable energy system for industrial internet of things model using fusion-AI, in: Shaw, R.N., Ghosh, A., Mekhilef, S., Balas, V.E. (Eds.), *Applications of AI and IOT in Renewable Energy*. Academic Press, pp. 107–128.
<https://doi.org/10.1016/B978-0-323-91699-8.00006-1>
43. Mean Squared Error | Definition, Formula, Interpretation and Examples [WWW Document], n.d. URL <https://www.geeksforgeeks.org/mean-squared-error/> (accessed 9.5.24).
44. Root-Mean-Square Error in R Programming [WWW Document], 2020. . GeeksforGeeks. URL <https://www.geeksforgeeks.org/root-mean-square-error-in-r-programming/> (accessed 9.5.24).
45. Frost, J., 2017. How To Interpret R-squared in Regression Analysis [WWW Document]. Statistics By Jim. URL <http://statisticsbyjim.com/regression/interpret-r-squared-regression/> (accessed 9.5.24).
46. Team, E.I.D., n.d. MAPE (Mean Absolute Percentage Error) [WWW Document]. Oracle Help Center. URL https://docs.oracle.com/en/cloud/saas/planning-budgeting-cloud/pfusu/insights_metrics_MAPE.html (accessed 9.5.24).
47. US Department of Commerce, N., n.d. Dew Point vs Humidity [WWW Document]. URL https://www.weather.gov/arx/why_dewpoint_vs_humidity (accessed 9.5.24).
48. 10 Hyperparameters to keep an eye on for your LSTM model — and other tips | by Kuldeep Chowdhury | Geek Culture | Medium [WWW Document], n.d. URL <https://medium.com/geekculture/10-hyperparameters-to-keep-an-eye-on-for-your-lstm-model-and-other-tips-f0ff5b63fcd4> (accessed 9.5.24).

